

RHEL 9 RHCE 讲义

RHEL 9 实操考试备考讲义

目录

[完成一场红帽实操考试的方法](#)

[Ansible 架构、配置与 Inventory](#)

[YAML、Playbook、Task 与模块](#)

[变量、Facts、注册结果与优先级](#)

[循环、条件、Handler、Block 与错误控制](#)

[文件、模板与 Jinja2](#)

[外部数据、敏感变量与 Vault](#)

[Include、Import、Role 与 Collection](#)

[自动化用户、软件包、仓库、服务与计划任务](#)

[自动化网络、防火墙与 SELinux](#)

[自动化存储、文件系统与挂载](#)

[Ansible 故障排除、验证与幂等性](#)

[RHCE 综合任务](#)

通用章 完成一场红帽实操考试的方法

红帽实操考试要求把系统从给定初态配置到可验证的目标终态。真正困难的不是记住某一条命令，而是在有限时间内持续回答三个问题：题目要求改变哪个对象，当前证据支持哪一步，怎样证明修改既生效又能够保持。本章建立 RHCSA 与 RHCE 共用的读题、操作和验收方法，并分别给出一页检查清单。

[概念] 目标终态（desired end state）是题目真正评分的系统状态；当前状态是操作前或某一步之后系统实际呈现的状态；持久配置决定状态能否在重启、服务重载或下一次自动化执行后继续成立。三者不能由同一条查询自动证明。

[概念] 对象、约束与证据构成实操题的最小模型。对象可能是用户、文件、服务、块设备、端口、Inventory 主机或受管节点；约束说明必须保留什么、不能改变什么；证据用于决定下一步并证明终态。

[操作语义] 调查命令读取对象而不改变目标状态；配置命令或 Playbook 改变对象；验证命令从配置、当前状态、功能、外部访问、持久性和幂等性中证明一个明确层次。

[操作语义] RHCSA 的主线是“手工修改单机终态”，RHCE 的主线是“用 Inventory、变量、模块和 Playbook 表达多机终态”。RHCE 的 `ok` 或 `changed` 只描述任务执行结果，不能替代受管节点上的功能验收。

[知识专题] 把题目改写成对象、状态、约束与证据

① [知识点] 先找评分对象，不从命令名开始

一道题可能写着“配置服务开机可用”，真正的评分对象至少包含软件包、配置文件、服务当前状态、启动配置、监听端口以及必要的访问链路。若只看到“服务”就立即执行 `systemctl start`，得到的只是一个局部当前状态。

读题时把每一项要求压缩成四列：对象是什么，当前状态未知在哪里，目标终态是什么，限制条件是什么。例如“保留现有数据扩展卷”中的“保留现有数据”是硬约束，它会排除重新格式化这条看似快速的路径。

② [知识点] 把模糊动词改写成可判定状态

“配置”“启用”“部署”“保证可访问”都不是单一动作。`enabled` 描述开机启动配置，`active` 描述当前运行；防火墙已放行不代表服务正在监听；目录存在不代表文件系统已经挂载；Playbook 没有 `failed` 不代表模板内容正确。

每个目标都应有能够判定对错的状态表达，例如：用户 UID 为指定值且属于给定附加组；挂载源为目标 UUID、当前已挂载且 `fstab` 可验证；服务处于 `active` 与 `enabled`，端口正在监听且本机或远端访问成功。

③ [边界] 已知条件不是永远可信的当前证据

题目提供的初始状态用于缩小调查范围，但操作过程中对象会互相影响。修改网络可能使远程连接中断；重新加载防火墙可能暴露遗漏；一个错误的 `fstab` 条目会影响启动。完成相关操作前仍应通过最小查询确认关键前提，尤其是设备身份、已有数据、目标主机范围和变量来源。

[Cheatsheet] 读题四问：对象是什么？当前缺什么证据？目标如何判定？哪些状态必须保留？把“配置成功”拆成配置、当前状态、功能与持久性。

[操作专题] 用最小调查建立可恢复的操作基线

① [操作点] 只查询会改变决策的字段

调查不是把所有命令运行一遍，而是为下一步选择证据。存储题先建立设备、文件系统、挂载和 LVM 映射；服务题先看配置语法、状态、日志与监听；RHCE 先确认项目目录、配置来源、Inventory 匹配、SSH 与提权。

可把调查结果保留在考试草稿中：记录题号、对象、当前证据、待验证项。不要在系统中创建会干扰评分的复杂跟踪平台，也不要把口令或 Vault 明文复制到草稿。

② [操作点] 优先选择最小且可重复的修改

最小修改只改变题目要求的对象。编辑配置前保留系统原有结构；已有记录时先查询再选择新增或修改；能够使用专用模块表达目标状态时，不用无条件执行的 `shell`。破坏性命令只能在设备身份、数据要求和题意都明确时使用。

一次完成一个可验证层次。服务配置可按“软件包 → 配置语法 → 服务当前与启动状态 → 监听 → 防火墙 → 功能”推进；存储可按“设备 → 容量对象 → 文件系统 → 当前挂载 → 持久配置 → 数据”推进。

③ [验证点] 每次修改立即留下局部证据

局部验证的价值是缩小故障范围。配置文件修改后先做语法或结构检查，再启动服务；写入 `fstab` 后在重启前运行静态验证和实际挂载；Playbook 先做语法检查、列出主机范围并限制到小组执行。

局部证据必须注明边界：`systemctl is-active` 证明当前运行，不证明开机启动；`findmnt` 证明当前挂载，不证明 `fstab`；`ansible-playbook --syntax-check` 只证明控制端能够解析结构，不证明远端任务可执行。

[Cheatsheet] 查询只看影响决策的字段；修改只推进一个目标层；每步立即验证并写清证明边界；破坏性操作前重新确认对象与保留要求。

[诊断专题] 卡题时保留现场，让下一条证据推进判断

① [诊断点] 先给失败定位层次

失败可以先粗分为输入或语法、对象不存在、权限或连接、容量或依赖、当前状态、持久配置、最终功能。这个分类不等于根因，却能决定下一条查询。命令不存在时先查帮助和软件包；服务失败看状态与日志；远端 unreachable 先查 Inventory、SSH 和提权，不应先改业务任务。

② [诊断点] 不用重复执行代替调查

重复运行相同命令只在幂等验证或确认瞬态问题时有意义。lvextend 报空间不足，应查询 VG 余量；端口新增提示已存在，应查询现有类型；模板任务失败，应查看变量、模板路径和渲染语法。错误信息是证据入口，不是要求“换几个参数试试”。

③ [诊断点] 暂时跳过时保存缺失证据

若一道题耗时过长，记录已经成立的层次、最后错误、尚未确认的对象和可能影响的其他题。尽量保留现场，不使用删除、重建或全局放宽权限来换取短暂成功。完成其他题后再回来，新的系统状态和剩余时间可能让判断更清晰。

[Cheatsheet] 现象 → 所在层 → 当前证据 → 一个假设 → 下一条查询；不要重复失败命令；跳题时记录“已证实、未证实、最后错误、影响范围”。

[操作专题] RHCSA：把单台主机验收到可重启终态

① [操作点] 按风险和依赖安排顺序

先确认身份、主机名、网络 and 仓库等基础条件，再处理用户、软件、服务、存储、SELinux、容器与恢复任务。高风险的磁盘、启动和网络修改应在证据充分时执行，并在修改后立即验证。题目之间共享对象时，记录后一次修改是否会覆盖前一题。

② [验证点] 一页 RHCSA 检查清单

- 网络地址、路由、DNS、主机名与名称解析符合题意；
- 仓库可用，指定软件包存在，服务配置语法正确；
- 用户 UID、主组、附加组、密码策略、权限、ACL 与 sudo 分别核对；
- 服务同时检查 active、enabled、监听、日志、firewalld 与最终访问；
- 分区、PV/VG/LV、文件系统、Swap、当前挂载与 fstab 分层检查；
- SELinux 保持 Enforcing，文件规则、当前标签、端口类型与 Boolean 使用最小入口；
- cron/at、时间同步、日志持久性和容器用户服务按题意验证；
- findmnt --verify、mount -a 等重启前检查没有未处理错误；
- 不依赖临时 chmod、临时标签、runtime-only 防火墙或手工启动状态；

- 最后从题目清单逐项复核，不以“命令都执行过”代替验收。

③ [边界] 重启是高价值但有前提的验证

重启能够暴露启动配置、挂载和服务依赖问题，但错误的网络或 `fstab` 也可能让系统难以恢复。应先完成静态验证、当前功能检查并保留恢复路径，再根据考试环境和题意决定是否重启。不能把未经检查的重启当作排错第一步。

[Cheatsheet] RHCSA：调查 → 手工修改 → 当前状态 → 功能 → 持久配置 → 交叉影响 → 最终清单；重启前先验证网络、`fstab` 与关键服务。

[操作专题] RHCE：从控制端语法推进到多节点幂等终态

① [操作点] 先证明执行边界

确认当前工作目录中的 `ansible.cfg` 是否生效，Inventory 是否解析到预期主机与组，连接用户、Python 与 become 是否可用。执行前使用 `--syntax-check`、`--list-hosts`、`--list-tasks`，必要时以 `--limit` 缩小第一轮范围。

② [验证点] 一页 RHCE 检查清单

- 项目目录、`ansible.cfg`、Inventory 与 requirements 文件位于题目要求路径；
- 主机组、嵌套关系、变量目录和 Vault 密码来源能够被当前项目读取；
- YAML 语法、模块 FQCN、字段类型、模板与变量名经过静态检查；
- 第一轮限制范围执行，区分 `failed`、`unreachable`、`changed`、`ok` 与 `skipped`；
- Handler 只由真实变更通知，错误控制没有掩盖必须失败的任务；
- 文件内容、所有者、模式、服务、端口、防火墙、SELinux、存储和挂载在受管节点验证；
- 不把控制节点文件存在误认为远端文件已经正确；
- 扩展到全部目标主机后再次核对 recap 与每组终态；
- 第二次执行不应出现无法解释的 `changed`，并再次检查远端功能；
- Vault 内容保持加密，输出和提交文件中没有明文秘密。

③ [验证点] 第二次执行只证明幂等性的一个维度

第二次执行没有 `changed`，说明模块认为目标状态已满足；若模板本身写错、Inventory 漏掉主机或验证对象选错，错误终态同样可能稳定不变。因此幂等检查必须和受管节点上的真实文件、服务、端口、挂载与业务访问组合使用。

[Cheatsheet] RHCE：配置/Inventory → 语法与主机范围 → 限制执行 → 远端终态 → 扩展执行 → 第二次执行 → 再验收；recap 不是最终功能证明。

[经典任务] 在混合任务清单中建立可验证的执行顺序

你接手一组尚未开始的考试任务：其中包含静态网络、软件仓库、共享目录、持久挂载、非标准 Web 服务，以及一个面向多组主机的自动化部署。部分任务共享服务、路径和端口；存储设备已有状态未知；自动化项目已提供若干变量文件。

目标终态：在不删除未知数据、不降低 SELinux 保护、不泄露敏感变量的前提下，为这些任务建立一条可执行顺序。每个任务都要说明操作前需要的证据、完成后最少验证层次、可能影响的其他任务，以及最终综合验收方法。

限制条件：不能用重建对象代替排错；不能以服务 active 代替外部访问；不能以 Playbook recap 成功代替远端终态；暂时跳过的任务必须能够恢复上下文。

验收要求：执行计划能够区分 RHCSA 与 RHCE 路径，包含重启前检查、幂等性检查、交叉影响检查和两份最终清单。

请先独立完成。参考分析与推荐路径从下一页开始。

[参考解答] 用证据门把高风险任务和依赖任务串起来

① [操作点] 读题拆解与依赖排序

先建立对象表：网络与名称解析影响仓库和远程连接；仓库影响软件包；存储设备身份决定能否写入；Web 访问依赖服务配置、监听、防火墙与 SELinux；自动化依赖控制端配置、Inventory、SSH、提权和变量。共享对象旁标记题号，避免后续操作覆盖已有终态。

推荐顺序是先做低风险基础调查，再完成网络、名称解析和仓库；随后处理身份、文件与服务；设备证据充分后处理存储；最后完成 SELinux 与外部功能闭环。RHCE 项目先做控制端静态检查和小范围执行，再扩展到所有受管节点。

② [验证点] 为每类任务设置证据门

网络任务至少验证连接配置、活动地址、路由、DNS 与必要连通；共享目录验证身份、组继承、权限与新文件行为；存储验证设备归属、容量对象、文件系统、当前挂载、持久条目与数据；Web 服务验证配置语法、active/enabled、监听、firewalld、SELinux 和最终请求。

自动化在执行前证明配置来源、主机范围和变量可解析；执行后在远端检查文件、服务与功能；第二次执行检查无法解释的变更。任何证据门失败时停留在该层调查，不删除已经正确的下层对象。

③ [诊断点] 卡题与最终收束

卡题记录“目标对象、已成立状态、最后错误、缺失证据、共享影响”。转去其他题时不留下临时 Permissive、全局权限放宽或半写入的关键配置。返回后从缺失证据继续，而不是重新执行全部命令。

最终按 RHCSA 与 RHCE 检查清单逐项验收。重启前先检查持久网络、`fstab` 和关键服务；是否重启取决于恢复条件。RHCE 在全量执行和第二次执行后仍要检查受管节点终态。

[Cheatsheet] 建对象表与共享影响 → 基础证据 → 低风险依赖 → 高风险存储/网络 → 服务完整链 → RHCE 小范围到全量 → 持久与幂等 → 最终清单。

[本章收束] 让每一步都能回答“改变了什么，证明了什么”

实操考试的稳定方法不是背一条固定流水线，而是持续维护对象、状态和证据之间的对应关系。调查只读取会影响决策的信息；修改只推进一个目标层；验证说明自己能证明什么、不能证明什么；卡题时保留现场并记录缺失证据。

RHCSA 最终把单机配置验收到当前、功能和持久状态；RHCE 还要证明主机范围、变量化、多节点终态与第二次执行。只要不把局部成功扩大成整体结论，就能把复杂任务拆成可恢复、可验证的小闭环。

第一章 Ansible 架构、配置与 Inventory

RHCE 的第一道证据链发生在控制节点：当前项目读取哪个 `ansible.cfg`、Inventory 匹配哪些主机、SSH 与提权使用什么身份。业务任务之前先证明执行边界，才能避免把 `unreachable` 误诊为模块错误。

[概念] 控制节点保存项目、Inventory、变量和内容；受管节点通常通过 SSH 接收临时模块执行，需要可用 Python 与适当提权。Ansible agentless 不等于没有远端前提。

[概念] Inventory 同时定义主机集合、组关系和少量连接变量；业务变量更适合 `group_vars/host_vars`。`pattern` 决定本次执行集合，错误匹配可能安静地遗漏主机。

[操作语义] `ansible-config dump --only-changed` 观察有效配置，`ansible-inventory --graph/--host` 展开 Inventory，`ansible <PATTERN> -m ping` 验证连接与 Python。

[操作语义] `ansible-doc` 查询模块和参数；ad hoc 适合一次性调查，Playbook 承担可重复目标状态。

[知识专题] 配置发现、项目边界与连接前提

① [知识点] 配置来源有优先顺序

`ANSIBLE_CONFIG` 可显式指定；当前目录、家目录和系统配置按规则搜索。项目中一个拼错或不可读的 `ansible.cfg` 可能导致使用另一份配置。`ansible --version` 会显示 config file。

② [知识点] inventory、remote_user 与 become 是独立参数

SSH 登录用户先连接，become 再在远端切换权限。能 ping 不证明 become 成功；需要 `-b` 的受控命令单独验证。

③ [验证点] 输出有效配置与版本

```
ansible --version
ansible-config dump --only-changed
ansible-inventory --graph
```

[Cheatsheet] 先证配置文件与 inventory；再证主机图、SSH、Python 和 become；连接成功不替代业务终态。

[操作专题] 编写静态 Inventory 并验证 pattern

① [操作点] 定义主机、组与嵌套组

```
[web]
servera ansible_host=192.0.2.11
serverb ansible_host=192.0.2.12

[production:children]
web
```

Inventory 名称是 `inventory_hostname`，连接地址可以不同。

② [操作点] 建立最小项目配置

```
[defaults]
inventory=./inventory
remote_user=devops
host_key_checking=True

[privilege_escalation]
become=True
```

不要把口令明文写进配置；使用题目提供的安全入口。

③ [验证点] 展开、匹配和连接

```
ansible-inventory --graph
ansible-inventory --host servera
ansible web --list-hosts
ansible web -m ansible.builtin.ping
ansible web -b -m ansible.builtin.command -a 'id -u'
```

[Cheatsheet] Inventory graph/host → pattern list-hosts → ping → become 身份；每步回答一个边界。

[诊断专题] 区分 unmatched、unreachable 与 privilege failure

① [诊断点] pattern 匹配 0 主机

先 `--list-hosts` 和 `inventory graph`，检查组名、inventory 路径和 children，不改 SSH。

② [诊断点] UNREACHABLE

使用 `-vvvv` 读取目标地址、用户、密钥和主机密钥错误；从控制节点手工 SSH 验证同一身份。

③ [诊断点] ping 成功但任务权限失败

检查 become、sudo 授权和 `ansible_become_user`，用 `id -u` 最小验证，不先给全局 NOPASSWD。

[Cheatsheet] 0 hosts 查 Inventory/pattern; unreachable 查 SSH; permission 查 become/sudo; 不要在错误层修改业务任务。

[经典任务] 建立可执行的多组主机项目

在指定目录创建 `ansible.cfg` 与静态 Inventory: `web`、`db` 两组归入 `production`, 使用 `devops` 连接并按题意提权。不得关闭主机密钥检查或写明文口令。验收配置来源、组图、每台 `hostvars`、`pattern`、SSH/Python 和 `root` 提权。

请先独立完成。参考解答从下一页开始。

[参考解答] 从配置发现到提权逐层验收

- ① [操作点] 在题目目录写相对 inventory 路径和 remote_user/become 配置。
- ② [操作点] 定义 web/db 与 production:children, 连接地址放 host vars。
- ③ [验证点] 运行 `ansible --version`、`config dump`、`inventory graph/host`、`--list-hosts`、`ping` 和 `command id -u -b`。若失败按 `unmatched/unreachable/become` 分层处理。

[Cheatsheet] `config file` → `inventory graph/host` → `list-hosts` → `ping` → `become id`; 不把一层成功扩大。

[本章收束] 先固定执行边界，再编写业务状态

Inventory 与配置是所有 Playbook 的作用域。只有明确控制节点读取的文件、目标主机集合、连接身份和提权能力，后续模块失败才有可解释的上下文。

第二章 YAML、Playbook、Task 与模块

Playbook 把目标主机、提权和一组任务写成可重复执行的 YAML。结构能通过解析只是第一道门，模块参数、目标主机、远端终态和第二次执行仍需独立验证。

[概念] YAML mapping 表达键值，sequence 表达列表；缩进属于结构，Tab 不用于缩进。一个 play 选择 hosts 并包含 tasks，每个 task 调用一个模块。

[概念] 专用模块表达目标状态并返回 changed；command 不经 Shell，shell 才解释管道/重定向。能用专用模块时不用无条件命令。

[操作语义] `ansible-playbook --syntax-check/--list-hosts/--list-tasks` 分别检查解析、主机和任务边界；`--check --diff` 在模块支持范围内预览。

[操作语义] 模块返回 ok/changed/failed/skipped；recap 总结执行，不证明远端业务功能。

[知识专题] YAML 结构、标量和模块参数

① [知识点] playbook 顶层是 play 列表

```
---
- name: Configure web
  hosts: web
  become: true
  tasks:
    - name: Install httpd
      ansible.builtin.dnf:
        name: httpd
        state: present
```

② [知识点] 布尔与权限模式要避免类型歧义

布尔用 `true/false`；文件 mode 常引用为 `'0644'`，避免 YAML 数值解释。含冒号、`#` 或 Jinja 表达式开头的字符串按需引用。

③ [验证点] 先让解析器和 `ansible-doc` 证明字段

`ansible-doc ansible.builtin.dnf` 查询字段，不从其他模块类推参数名。

[Cheatsheet] 顶层 play 列表；play 含 hosts/tasks；task 只调用一个模块；布尔 `true/false`、mode 引用；字段查 `ansible-doc`。

[操作专题] 安装、配置并启动服务的最小 Playbook

① [操作点] 用目标状态模块

```
- name: Deploy web
  hosts: web
  become: true
  tasks:
    - ansible.builtin.dnf:
        name: httpd
        state: present
    - ansible.builtin.copy:
        content: "exam\\n"
        dest: /var/www/html/index.html
        mode: '0644'
    - ansible.builtin.service:
        name: httpd
        state: started
        enabled: true
```

② [验证点] 静态、限制执行与远端终态

```
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml --list-hosts
ansible-playbook site.yml --limit servera
ansible web -b -m command -a 'systemctl is-enabled httpd'
```

再从受管节点或客户端检查文件与 HTTP。

③ [验证点] 第二次执行解释 changed

第二次无 changed 是幂等证据；仍有 changed 时逐 task 查无条件 command、动态内容或错误 changed_when。

[Cheatsheet] syntax/list → limit 执行 → recap → 远端文件/服务/HTTP → 全量 → 第二次执行；recap 不替代终态。

[诊断专题] 区分 YAML、模块、连接和远端失败

① [诊断点] syntax error

根据行列号检查上一层缩进、冒号和列表标记，不只盯报错行。

② [诊断点] unsupported parameters

用 `ansible-doc` 核对 FQCN 与当前 collection 版本字段，不把其他模块参数复制过来。

③ [诊断点] command 与 shell 选错

command 中 `|`、`>` 不会由 Shell 解释；若确需 Shell 使用 `shell` 并确保幂等/引用，优先寻找专用模块。

[Cheatsheet] 解析错查 YAML；参数错查 `ansible-doc`；`unreachable` 回连接层；远端 `failed` 读 module msg；命令行为先选专用模块。

[经典任务] 用 Playbook 部署可重复 Web 终态

在 web 组安装 httpd、部署指定首页并启用服务。第一次先限制到一台，随后扩展全组，第二次执行应无法解释的 changed。不得用 shell 模拟包、文件和服务模块。验收语法、主机范围、recap、远端文件、active/enabled 与 HTTP。

请先独立完成。参考解答从下一页开始。

[参考解答] 用三个专用模块表达终态

① [操作点] 使用 `ansible.builtin.dnf`、`copy`、`service` 分别表达包、文件和服务。

② [验证点] `syntax-check`、`list-hosts` 后 `limit` 执行；远端检查 `rpm`、`stat`、`systemctl` 与 `curl`。

③ [验证点] 全组执行后第二次运行，逐项解释 `changed`；最终再次 HTTP 验收。

[Cheatsheet] 专用模块 → 静态边界 → 小范围 → 远端终态 → 全量 → 第二次执行。

[本章收束] 执行成功必须落到远端目标状态

YAML 和模块把意图结构化，但只有目标主机范围正确、模块字段有效、远端对象符合要求且重复执行稳定，Playbook 才真正完成。

第三章 变量、Facts、注册结果与优先级

变量把同一自动化逻辑映射到不同主机；Facts 描述受管节点，register 保存某个任务的运行结果。正确性取决于变量来源、数据类型、主机作用域和实际返回结构。

[概念] 变量可来自 Inventory、group_vars、host_vars、play vars、vars_files、Facts、register、set_fact 和 extra vars；高优先级覆盖低优先级，但考试重点是减少冲突而非背完整表。

[概念] Facts 是每台主机的数据；magic variables 如 inventory_hostname、groups、hostvars 描述 Inventory 上下文。register 结果是字典，字段随模块与循环改变。

[操作语义] `ansible-inventory --host` 观察 Inventory 变量；`ansible -m setup -a filter=...` 查询 Facts；`debug: var=` 显示对象结构而不套 Jinja 双括号。

[知识专题] 变量来源与作用域

① [知识点] 变量来源与作用域

group_vars/all 作用全体，组文件作用组，host_vars 作用单主机。文件名必须与 Inventory 名称对应；把同一变量在多层重复定义会让覆盖难以审计。

② [知识点] Facts 与 magic variables

`ansible_facts['distribution']` 等来自远端采集；`inventory_hostname` 不要求 DNS 可解析，`ansible_host` 是连接地址。`groups['web']` 是主机名列表，`hostvars[h]` 访问另一主机变量。

③ [验证点] 先看类型和最终值

使用 `debug: var=myvar`、type_debug 过滤器和 `inventory --host`。字符串 `false` 与布尔 `false` 不同，when 中尤其危险。

[Cheatsheet] 变量来源与作用域；Facts 与 magic variables；先看类型和最终值

[操作专题] 组织 group_vars 与 host_vars

① [操作点] 组织 group_vars 与 host_vars

将共同值放 `group_vars/all.yml`，组差异放 `group_vars/web.yml`，主机例外放 `host_vars/servera.yml`；复杂数据使用 YAML 列表/字典。

② [操作点] 注册并读取结果

command 任务 register: check 后读取 check.rc/stdout/stderr/changed；循环 register 的单项结果位于 results。不要假设所有模块都有 stdout。

③ [验证点] 按主机显示并应用差异

先 debug 代表主机的变量/Facts，再 limit 执行；最终在每组受管节点检查差异化文件和服务。

[Cheatsheet] 组织 group_vars 与 host_vars；注册并读取结果；按主机显示并应用差异

[诊断专题] 变量未定义

① [诊断点] 变量未定义

用 -vvv、debug 和 inventory --host 查拼写、文件名、作用域和条件分支，不用 default 隐藏必需变量缺失。

② [诊断点] 字典字段不存在

先 debug 完整 register/Facts 结构，循环结果尤其查看 results；不要从示例版本推定字段。

③ [诊断点] 值正确但类型错误

用 type_debug 检查引号导致的字符串/布尔/整数差异，修正数据源而非在每个 task 强制转换。

[Cheatsheet] 变量未定义；字典字段不存在；值正确但类型错误

[经典任务] 用组变量和 Facts 部署差异配置

web 与 db 组使用不同包、端口和模板值；RHEL 9 才执行目标任务。不得复制两份 Playbook。验收最终变量、Facts 条件、每组文件和服务。

请先独立完成。参考解答从下一页开始。

[参考解答] 把差异放入数据并验证返回结构

① [操作点] 在 `group_vars` 定义 `service_package/service_port`，用 Facts 条件限制 RHEL 9。

② [操作点] 注册验证命令并根据 `rc` 显示明确结果。

③ [验证点] `inventory --host`、`debug type`、`limit` 与远端文件/监听交叉检查。

[Cheatsheet] 数据源 → 类型 → host 最终值 → limit 执行 → 每组终态。

[本章收束] 变量只有在主机上下文中才有确定值

把共享值、组差异和主机例外放在最小层次；Facts 与 `register` 先观察结构再引用。这样优先级不再是猜测，而是可查询的数据合并结果。

第四章 循环、条件、Handler、Block 与错误控制

控制结构用于对数据集合和主机差异重复表达目标，同时让变更、失败和恢复保持可解释。循环不是复制 task，Handler 不是无条件重启，错误控制也不能把必须失败的状态吞掉。

[概念] loop 为每项设置 item，可用 loop_control 改名；when 对每台主机和每个 item 求值。Handler 只在通知 task 返回 changed 时排队，默认在 play 后部运行。

[概念] block 组织任务并可配 rescue/always；failed_when 和 changed_when 改写结果语义，必须基于可靠返回字段。

[操作语义] notify 触发命名 Handler，meta: flush_handlers 提前运行；assert 把前置条件写成明确失败。

[知识专题] 循环数据应保持结构

① [知识点] 循环数据应保持结构

字典列表让 item.name/item.state 明确。避免平行列表和基于索引拼接。loop_control.label 只改善输出，不改变数据。

② [知识点] when 不使用 Jinja 定界符

when: ansible_facts['os_family'] == 'RedHat' 是表达式；字符串布尔需要显式类型。多个条件列表通常 AND。

③ [知识点] Handler 合并通知

同一 Handler 被多次通知通常只运行一次；通知 task 未 changed 则不运行。配置变化才重启是幂等关键。

[Cheatsheet] 循环数据应保持结构；when 不使用 Jinja 定界符；Handler 合并通知

[操作专题] 循环创建对象

① [操作点] 循环创建对象

使用 user/package 等专用模块遍历字典；给 loop_var 命名避免 include 嵌套 item 冲突。

② [操作点] 根据 register 定义结果

command 探测可用 changed_when: false，按 rc 定义 failed_when；但专用查询模块优先。

③ [验证点] 检查 changed 与 Handler 时点

第一轮配置变化应通知，第二轮无变化不应重启；必要时 flush 后再做依赖 Handler 的功能验证。

[Cheatsheet] 循环创建对象；根据 register 定义结果；检查 changed 与 Handler 时点

[诊断专题] ignore_errors 掩盖失败

① [诊断点] ignore_errors 掩盖失败

它继续执行但主机仍有失败语义，可能让后续使用无效状态。合法替代是明确 failed_when 或 block/rescue。

② [诊断点] Handler 未运行

查通知 task 是否 changed、notify 名称、play 是否在失败前到达 Handler，必要时 flush。

③ [诊断点] 每轮都 changed

查 command/shell、时间戳内容、changed_when 和模块目标值；第二次执行定位具体 task。

[Cheatsheet] ignore_errors 掩盖失败；Handler 未运行；每轮都 changed

[知识专题] 控制结构的输出与边界检查

① [参数点] loop_control 不改变业务数据

label 只缩短终端输出，loop_var 改变循环变量名，index_var 保存索引。调试时仍应检查原始 item 结构，不能因为 label 美观就认为数据完整。

② [验证点] failed_when 与 changed_when 必须可复核

条件应引用稳定的 rc、模块字段或明确输出，不匹配本地化人类文本。执行后用 debug: var=result 抽查被改写的任务结果，确保真实失败没有被标成成功。

③ [边界] rescue 不是事务回滚

block 中前一任务已经改变的远端状态不会自动撤销。rescue 必须显式恢复需要恢复的对象，always 只适合清理、记录和无条件收束。

[Cheatsheet] loop_control 管输出/变量名；结果改写必须基于稳定字段；rescue 需显式恢复，不是自动回滚。

[经典任务] 批量创建服务并只在变化时重启

从字典列表安装多个包并部署配置；仅配置改变时重启对应服务。对不满足内存条件主机明确跳过，并在失败时输出可诊断信息。

请先独立完成。参考解答从下一页开始。

[参考解答] 让数据、条件和 Handler 对齐

- ① [操作点] loop 字典调用 package, 模板 task notify Handler。
- ② [操作点] when 使用 Facts; block/rescue 处理受控异常。
- ③ [验证点] 第一轮观察 changed/handler, 第二轮确认无重启, 远端查服务。

[Cheatsheet] 结构数据 → when → 变更通知 → Handler → 第二次执行。

[本章收束] 控制结构应提高可解释性

循环减少重复, 条件限定适用主机, Handler 绑定真实变更, block 保留失败边界。任何结构都应让 recap 和远端终态更清楚, 而不是隐藏错误。

第五章 文件、模板与 Jinja2

文件自动化首先选择所有权边界：完整内容由 `copy/template` 管，单行由 `lineinfile` 管，受标记区块由 `blockinfile` 管。模板把变量渲染成每台主机的完整文件，错误内容也可能幂等。

[概念] `file` 管路径状态/身份/模式，`copy` 分发静态内容，`template` 渲染 Jinja2，`lineinfile` 维护一行，`blockinfile` 维护带 `marker` 的区块。选择过细工具修改完整受管文件会留下未知旧内容。

[概念] Jinja2 `{{ }}` 输出表达式，`{% %}` 控制结构，`{# #}` 注释。模板在控制节点渲染，Facts/变量来自目标主机上下文。

[操作语义] `template` 的 `validate` 在替换前用临时文件运行校验命令；成功后原子替换并 `notify Handler`。`diff` 与远端语法/功能共同验收。

[知识专题] 按所有权选择文件模块

① [知识点] 按所有权选择文件模块

完整文件归项目所有用 `template/copy`；只拥有一行用 `lineinfile`；只拥有区块用 `blockinfile`。`file state=touch` 每次改变时间，不适合只保证存在。

② [知识点] 模板变量必须处理缺失与类型

用 `mandatory/assert` 处理必需数据；`default` 只用于合法默认。过滤器如 `join`、`sort`、`to_nice_yaml` 改变输出，要检查目标格式。

③ [验证点] 内容、元数据和消费者分层

`stat` / `slurp/checksum` 查文件，应用 `-t` 或 `validate` 查语法，服务 `active/HTTP` 查功能。

[Cheatsheet] 按所有权选择文件模块；模板变量必须处理缺失与类型；内容、元数据和消费者分层

[操作专题] 渲染差异配置

① [操作点] 渲染差异配置

模板使用 `inventory_hostname`、组变量和 Facts；`for` 循环生成重复行，`if` 只表达真正主机差异。保持缩进与换行符合目标格式。

② [操作点] 安全替换并通知

`template` 设置 `owner/group/mode`、`backup`（按需）、`validate`，并 `notify restart Handler`；只有内容/元数据变化才 `changed`。

③ [验证点] check/diff 与远端语法

先 `--check --diff` 预览支持的变化，再 `limit` 执行；远端运行应用校验并查看渲染内容。

[Cheatsheet] 渲染差异配置；安全替换并通知；check/diff 与远端语法

[诊断专题] 模板变量未定义

① [诊断点] 模板变量未定义

定位变量作用域和拼写，debug 数据结构；不在模板各处 default 空字符串。

② [诊断点] 渲染成功但服务失败

控制端 Jinja 合法不等于目标配置语法合法；使用 `validate` 和远端日志。

③ [诊断点] lineinfile 每次 changed

检查 `regexp` 是否能匹配写入后的 `line`，避免插入重复行；必要时选择 `template`。

[Cheatsheet] 模板变量未定义；渲染成功但服务失败；lineinfile 每次 changed

[知识专题] 模板渲染、校验与换行细节

① [参数点] validate 命令使用占位符

`validate: '/usr/sbin/sshd -t -f %s'` 让模块把临时文件路径代入 `%s`。命令不经 Shell，因此管道与重定向不能直接使用；校验工具必须能读取临时路径。

② [知识点] Jinja 空白会改变真实配置

`trim_blocks`、`lstrip_blocks` 和 `-{%` 等控制空白。多一空行通常无害，但 YAML、sudoers、hosts 或严格配置中的缩进与末尾换行可能改变语义，必须查看 `--diff` 和远端文件。

③ [验证点] 模板依赖清单

模板引用的每个变量都应能从 `defaults`、`group_vars`、`host_vars` 或 `Facts` 追溯；修改模板后回归所有消费主机组，而不只测试当前 `limit` 主机。

[Cheatsheet] `validate` 用 `%s` 临时路径；空白控制可能改语义；diff 后回归所有模板消费者。

[经典任务] 为不同主机生成服务配置并安全重载

使用一个模板按 web 组变量生成监听端口和后端列表，所有者/模式固定；配置校验成功且内容变化时才重载服务。验收 diff、远端内容、语法、Handler、监听和第二次执行。

请先独立完成。参考解答从下一页开始。

[参考解答] 让模板成为完整文件真相

- ① [操作点] 在 j2 中使用明确变量、循环与最小条件。
- ② [操作点] template 设元数据和 validate, notify reload Handler。
- ③ [验证点] check/diff、limit、远端语法/监听, 第二次无 changed。

[Cheatsheet] 数据 → 模板 → validate → 原子替换 → Handler → 远端功能/幂等。

[本章收束] 文件模块的选择决定维护边界

完整文件、单行和区块应由不同模块承担。Jinja 只负责渲染, validate 与远端功能证明内容可用; 第二次执行证明表达稳定。

第六章 外部数据、敏感变量与 Vault

外部变量把用户、服务和环境数据从任务逻辑中分离；Vault 只加密静态内容，不自动隐藏运行时输出。正确流程同时保证数据结构可解析、秘密在磁盘中加密、运行时可解密且日志不泄露。

[概念] `vars_files` 显式加载数据文件，`group_vars/host_vars` 按 Inventory 自动加载。YAML 列表适合对象集合，字典适合按名称索引；稳定 schema 比在 task 中兼容多种形状更可靠。

[概念] Vault 可加密整个文件或单个字符串。vault ID 允许多个密码来源；密码文件本身必须受保护且不进入发布内容。

[操作语义] `ansible-vault create/edit/view/encrypt/decrypt/rekey/encrypt_string` 管密文；`--vault-password-file` 或 `--vault-id` 提供运行时秘密。`no_log: true` 只在必要 task 抑制输出。

[知识专题] 外部数据要有明确 schema

① [知识点] 外部数据要有明确 schema

批量用户可定义 `name/groups/password_hash` 等键；Playbook 用 `assert` 验证必需键，避免部分对象执行到中途才失败。

② [知识点] Vault 密文仍是内容源

编辑用 `ansible-vault edit`，不先 `decrypt` 留明文临时文件。`rekey` 更换密码保持数据加密；`decrypt` 是明确导出明文的高风险操作。

③ [验证点] 检查文件头和安全输出

Vault 文件以 `$ANSIBLE_VAULT;` 开头；`view` 能按授权读取。运行后检查目标终态，不打印秘密值。

[Cheatsheet] 外部数据要有明确 schema；Vault 密文仍是内容源；检查文件头和安全输出

[操作专题] 加载外部用户列表

① [操作点] 加载外部用户列表

`vars_files` 加载 YAML，`loop` 遍历用户字典；密码字段应是目标模块所需哈希，不把明文直接传给 `user.password`。

② [操作点] 创建和改密 Vault

使用 `create/edit`；密码轮换用 `rekey old/new vault-id`。密码文件权限收紧并排除版本控制。

③ [验证点] 语法、解密与远端对象

先 vault view 和 playbook syntax-check, 再 limit; 远端用 getent/id 验证用户, 不回显哈希。

[Cheatsheet] 加载外部用户列表; 创建和改密 Vault; 语法、解密与远端对象

[诊断专题] no vault secrets found

① [诊断点] no vault secrets found

检查当前项目是否收到正确 vault-id/password file, 文件是否被错误 decrypt 或 vault ID 不匹配。

② [诊断点] 秘密出现在日志

给最小敏感 task 设置 no_log, 并审查 debug、失败消息与注册变量; 不能靠删终端历史作为修复。

③ [诊断点] 数据结构与 loop 不匹配

debug 仅显示非敏感键/类型, 使用 assert; 修正 schema, 不在任务中叠加复杂兼容。

[Cheatsheet] no vault secrets found; 秘密出现在日志; 数据结构与 loop 不匹配

[知识专题] Vault ID、文件权限与日志泄露面

① [参数点] 多 Vault 使用 label

`--vault-id dev@prompt --vault-id prod@/path/key` 将密文 label 与密码来源绑定。运行错误时先确认密文头中的 vault ID 和命令提供的 label, 不按文件名猜密码。

② [边界] 密码文件不是 Vault

密码文件通常是解密密钥本身, 必须 `0600`、排除版本控制并按题目路径管理; 把它再放进同一 Vault 会形成无法启动的循环依赖。

③ [验证点] 失败输出也可能泄密

模块失败的 invocation、register、debug 和 callback 均可能包含参数。对传递秘密的最小 task 使用 `no_log`, 同时避免随后 debug 整个注册对象。

[Cheatsheet] 多 Vault 核对 label; 密码文件 0600 且不入库; 敏感 task 与后续 register 输出一一起审计。

[经典任务] 使用加密变量批量创建账号

下载/提供的用户列表保存在外部 YAML，密码哈希放 Vault；按组变量选择附加组。密码文件路径由题目指定且不得泄露。验收 Vault 保持加密、Playbook 可运行、用户/组正确且输出无秘密。

请先独立完成。参考解答从下一页开始。

[参考解答] 以 schema 和 Vault 边界驱动用户任务

- ① [操作点] 验证用户列表必需键, Vault 保存哈希与敏感值。
 - ② [操作点] vars_files 加载, user 模块 loop 创建; 敏感 task no_log。
 - ③ [验证点] vault view/syntax/limit, 远端 getent/id, 最终确认源文件仍加密。
- [Cheatsheet] schema/assert → Vault → 安全加载 → user loop → 远端身份 → 密文与日志审计。

[本章收束] 加密不等于不会泄露

Vault 保护静态文件, no_log 控制任务输出, 文件权限和版本控制保护密码入口。三层同时成立, 外部数据才既可维护又安全。

第七章 Include、Import、Role 与 Collection

内容拆分必须保留执行边界。import 在解析期静态展开，include 在运行期动态选择；Role 用固定目录把 tasks、handlers、templates、files 和变量组织为可复用接口；Collection 提供 FQCN、Role 和插件版本。

[概念] include_tasks 动态执行并可按 loop/when 选择；import_tasks 静态展开，适合固定结构和 list-tasks。import_playbook 只能在 playbook 顶层。

[概念] Role defaults 是低优先级可覆盖接口，vars 更强不适合作为用户配置入口。Collection requirements 锁定依赖来源/版本，FQCN 避免模块名冲突。

[操作语义] `ansible-galaxy role init` 创建 Role 骨架，`ansible-galaxy collection install -r requirements.yml` 安装依赖；`ansible-doc -t role` / 模块文档查接口。

[知识专题] 动态与静态复用

① [知识点] 动态与静态复用

import 在解析时展开，标签/list-tasks 可见；include 在运行时根据变量与循环决定。不要只背先后，按是否需要运行时选择。

② [知识点] Role 目录和变量接口

tasks/main.yml 是入口，handlers/templates/files 自动按 Role 路径解析；defaults 提供可覆盖默认值，meta 声明依赖。

③ [验证点] 依赖与路径可复现

requirements 文件进入项目；离线/考试材料按给定 tar/路径安装，`ansible-galaxy collection list` 核对实际版本。

[Cheatsheet] 动态与静态复用；Role 目录和变量接口；依赖与路径可复现

[操作专题] 把单文件重构为 Role

① [操作点] 把单文件重构为 Role

移动任务、Handler、模板和文件，不改变 notify 名称与变量语义；把环境差异转为 defaults 或调用时 vars。

② [操作点] 安装并使用 Collection/System Role

requirements 指定 name/source/version; Playbook 用 FQCN 和 `roles:` 调用, 阅读 Role README/defaults 获取变量 schema。

③ [验证点] 语法、任务图与远端终态

syntax-check、list-tasks/tags、limit 执行; Role 重构前后远端终态与第二次执行一致。

[Cheatsheet] 把单文件重构为 Role; 安装并使用 Collection/System Role; 语法、任务图与远端终态

[诊断专题] 找不到 Role/Collection

① [诊断点] 找不到 Role/Collection

检查 roles_path/collections_paths、项目目录、requirements 安装位置和名称空间, 不复制内容到随机路径。

② [诊断点] 变量无法覆盖

检查是否错误放在 role vars 而非 defaults, 以及调用层变量名/schema。

③ [诊断点] include 标签行为意外

动态 include 的标签不会自动以相同方式传播到内部 task; 按文档使用 apply 或给内部任务标签。

[Cheatsheet] 找不到 Role/Collection; 变量无法覆盖; include 标签行为意外

[知识专题] Role 接口、依赖与标签传播

① [知识点] defaults 是公开接口, 不是所有变量仓库

调用者需要覆盖的值进入 defaults; Role 内部常量可放 vars, 但过强优先级会阻止环境差异。变量名加 Role 前缀可减少多 Role 冲突。

② [参数点] requirements 可同时描述 Role 与 Collection

实际项目可分 roles/requirements.yml 与 collections/requirements.yml, 也可按工具支持格式组织。离线 tarball 的 source 与版本要保持题目给定 lineage, 不得擅自换最新版。

③ [验证点] 重构前后比较任务与 Handler

`--list-tasks`、tags、远端 diff 和第二次执行共同证明移动内容没有丢失 Handler、模板路径或变量覆盖。

[Cheatsheet] defaults 暴露接口; requirements 固定真实依赖; 重构用任务图、远端 diff 与幂等证明等价。

[经典任务] 把 Web Playbook 重构为可复用 Role

创建 `web_role`，包含包、模板、Handler 和服务；默认端口可由组变量覆盖。通过 `requirements` 安装题目 Collection，并用新 Playbook 调用。验收依赖、任务图、两组差异、Handler 和幂等性。

请先独立完成。参考解答从下一页开始。

[参考解答] 保留行为地移动内容

① [操作点] role init, 移动 tasks/handlers/templates/files, 接口值放 defaults。

② [操作点] requirements 安装 Collection, 以 FQCN 调用。

③ [验证点] syntax/list-tasks、limit、全量、远端终态、第二次执行。

[Cheatsheet] 依赖安装 → Role 接口 → 静态边界 → 远端等价 → 幂等。

[本章收束] 复用的价值是稳定接口而非更多目录

选择 include/import 的时点语义, Role 用 defaults 暴露最小接口, Collection 用 requirements 固定依赖。重构完成必须证明远端行为未改变。

第八章 自动化用户、软件包、仓库、服务与计划任务

本章把 RHCSA 手工状态映射为专用模块参数。自动化重点是声明最终用户、仓库、包、服务和 cron，而不是远程执行对应命令；变量数据驱动多对象，Handler 只响应配置变化。

[概念] user/group/authorized_key 管身份，dnf/package/yum_repository 管软件，service/systemd_service 管当前与启动状态，cron 管周期声明。每个模块的 state 对应目标终态。

[概念] 模块幂等依赖稳定输入：随机密码盐、动态时间戳、无 regexp 的追加文本都会制造持续 changed。

[操作语义] 使用 FQCN；password 传哈希，groups 配合 append；enabled 与 state 分别表达启动和当前服务。

[知识专题] 身份模块参数边界

① [知识点] 身份模块参数边界

user.groups 默认可替换附加组，append: true 才追加；remove 要明确是否删除家目录。authorized_key 的 exclusive 对循环逐项使用可能互相删除。

② [知识点] 仓库与包分层

yum_repository 创建 repo 配置，rpm_key 管 key，dnf 安装包。仓库文件存在后仍要 makecache/包事务证据。

③ [验证点] 服务双态与 cron 身份

service 的 started/enabled 分别验证；cron 的 user、时间字段和 job 必须与题意一致，远端检查 crontab/产物。

[Cheatsheet] 身份模块参数边界；仓库与包分层；服务双态与 cron 身份

[操作专题] 用外部列表创建用户

① [操作点] 用外部列表创建用户

loop 字典调用 group/user/authorized_key；密码哈希从 Vault，no_log 限制敏感 task。

② [操作点] 配置仓库、包和服务

先 key/repo，再 dnf；模板 notify Handler，service 保证 started+enabled。

③ [验证点] 模块结果与远端对象

第二次执行无 `changed`；`getent/id`、`dnf repolist/rpm`、`systemctl`、`crontab` 和实际服务功能。

[Cheatsheet] 用外部列表创建用户；配置仓库、包和服务；模块结果与远端对象

[诊断专题] 用户附加组丢失

① [诊断点] 用户附加组丢失

检查 `groups` 是否在未 `append` 情况下替换全部列表；修正数据为完整目标或 `append`。

② [诊断点] 服务每次重启

检查 `Handler` 是否被动态模板/无条件 `changed task` 每次通知。

③ [诊断点] `cron` 存在但不执行

远端查 `user`、绝对路径、环境、`crond` 日志和产物；模块 `changed=0` 只证明条目稳定。

[Cheatsheet] 用户附加组丢失；服务每次重启；`cron` 存在但不执行

[知识专题] 模块参数的替换、追加与删除语义

① [参数点] `user` 模块表达最终账号而非 `useradd` 命令

`state`、`uid`、`group`、`groups`、`append`、`shell`、`password_expire_max` 等共同描述目标。删除账号时 `remove: true` 会删除家目录，必须由题意明确。

② [参数点] `dnf state` 的范围

`present` 保证安装，`latest` 会随仓库元数据升级并可能持续改变发布结果，`absent` 删除。只有题目要求最新时使用 `latest`，并阅读依赖事务。

③ [验证点] `authorized_key` 与 `sudoers` 仍需功能测试

公钥文件存在后从控制节点实际 SSH；`sudoers` 由 `template/copy` 部署时用 `visudo -cf %s validate`，再以目标用户执行允许与拒绝命令。

[Cheatsheet] `user` 删除/组语义要显式；`dnf latest` 只按题意；公钥与 `sudoers` 最终做真实认证/授权测试。

[经典任务] 用变量为多组主机配置基础服务

从外部用户数据创建组/用户/公钥，配置指定仓库和包，部署服务并只在配置变化时重启，创建周期任务。验收每组数据差异、秘密安全、服务双态、cron 产物和第二次执行。

请先独立完成。参考解答从下一页开始。

[参考解答] 把手工命令映射为专用模块

- ① [操作点] `group/user/authorized_key loop`; Vault 哈希。
- ② [操作点] `rpm_key/yum_repository/dnf/template/Handler/service`。
- ③ [操作点] `cron` 声明; 远端 `getent/rpm/systemctl/crontab/功能`, 第二次执行。

[Cheatsheet] 数据 → 身份 → 仓库/包 → 配置/Handler/服务 → `cron` → 远端/幂等。

[本章收束] 自动化对象要以 state 而非命令表达

专用模块让 Ansible 比较当前和目标; 变量提供差异, Handler 绑定真实变化。recap 之外仍需在受管节点证明身份、包、服务和调度功能。

第九章 自动化网络、防火墙与 SELinux

网络、安全与 SELinux 自动化要求同时表达当前和持久状态，并避免批量断连。模块或 RHEL System Role 描述目标连接；firewalld immediate/permanent、SELinux fcontext/port/boolean 分别对应不同策略对象。

[概念] 网络变更可能切断 Ansible 控制通道，应先 limit、保留旧连接和控制台恢复路径。RHEL network System Role 以结构化变量表达连接集合。

[概念] firewalld 的 permanent 与 immediate 分别写持久和 runtime；SELinux fcontext 建规则后仍要 restorecon，端口和 Boolean 是独立入口。

[操作语义] `ansible.posix.firewalld`、`community.general.sefcontext/seport` 与 `ansible.posix.seboolean`（以安装 collection 文档为准）表达状态；command 只补没有模块的恢复动作。

[知识专题] 网络 role 输入必须按节点变量化

① [知识点] 网络 role 输入必须按节点变量化

每台主机地址/网关/DNS 放 host/group vars，不在 task 中按主机名堆 when。应用连接前验证设备名与现有连接。

② [知识点] firewalld 双态

`permanent: true` 写持久；`immediate: true` 同时应用 runtime，前提 firewalld 运行。zone/service/port 必须匹配。

③ [知识点] SELinux 三类状态

sefcontext 管路径规则，restorecon 应用当前标签；seport 管协议/端口类型；seboolean 的 persistent 管持久值。

[Cheatsheet] 网络 role 输入必须按节点变量化；firewalld 双态；SELinux 三类状态

[操作专题] 小范围应用网络

① [操作点] 小范围应用网络

syntax/check（role 支持范围）、limit 单主机，等待连接恢复并重新 gather facts；再扩全组。

② [操作点] 部署非标准 Web 安全状态

template 配置端口，firewalld service/port 双态，sefcontext+restorecon，seport，服务 Handler。

③ [验证点] 远端终态和外部访问

nmcli/ip、firewall-cmd 双查询、semanage/lsc -Z/getsebool、ss 与外部 curl；第二次执行。

[Cheatsheet] 小范围应用网络；部署非标准 Web 安全状态；远端终态和外部访问

[诊断专题] 网络 task 后 unreachable

① [诊断点] 网络 task 后 unreachable

从控制台/旧地址确认 profile 与 route，检查 Ansible 是否等待重连；不要在全组重复错误连接。

② [诊断点] firewalld 每次 changed

检查 immediate/permanent、zone 和 module 版本字段，避免 reload command 每次执行。

③ [诊断点] 标签规则存在但访问仍拒绝

确认 restorecon 已应用、目标类型/端口/Boolean 与 AVC；规则存在不等于当前标签。

[Cheatsheet] 网络 task 后 unreachable；firewalld 每次 changed；标签规则存在但访问仍拒绝

[知识专题] 网络批量变更的串行与恢复策略

① [边界] serial 降低影响但不修复错误

play 设置 `serial: 1` 可逐台应用网络，配合 `max_fail_percentage` 控制停止条件。第一台失败时立即停止并调查，不能依赖 serial 自动回滚连接。

② [验证点] 等待重连要验证新地址

网络 Role 完成后可用 `wait_for_connection`，但成功只证明 Ansible 重新连接；继续运行 `ip route get`、`getent` 和目标协议测试，确认连接不是经错误备用路径恢复。

③ [边界] SELinux module 与 restorecon 分工

规则模块只保证 policy mapping，当前对象仍需恢复标签。restorecon command 可先用 `-n` 预览差异，再在规则确定后应用并用 `matchpathcon / ls -Z` 对比。

[Cheatsheet] 网络用 serial/停止条件降低批量风险；重连后验地址/路由；fcontext 规则与 restorecon 当前标签分开。

[经典任务] 自动部署自定义目录与端口 Web 服务

按主机变量配置网络；部署 httpd 自定义目录和 8080，配置 firewalld 当前/持久、SELinux 文件规则和端口类型。不得 Permissive。先一台再全组，验收连接恢复、安全三层、监听、外部 HTTP 和幂等性。

请先独立完成。参考解答从下一页开始。

[参考解答] 按风险顺序应用网络与安全

① [操作点] network role limit, 一台恢复连接并验证路由/DNS。

② [操作点] 模板/Handler、firewalld 双态、sefcontext+restorecon、seport。

③ [验证点] 远端命令与外部 curl, 全组后第二次执行。

[Cheatsheet] 网络小范围/重连 → 服务 → 防火墙双态 → SELinux 规则/当前标签/端口 → 外部功能/幂等。

[本章收束] 自动化仍要尊重系统状态分层

模块把目标状态写成参数，但连接风险、runtime/permanent 和规则/当前标签的区别没有消失。先小范围、再远端证据、最后全量和幂等。

第十章 自动化存储、文件系统与挂载

存储自动化必须面对每台主机设备条件不同和写操作不可逆。模块能保证对象 state，却不能判断题目中的设备是否安全；设备证据、容量来源、文件系统和 mount 双态仍需逐层验证。

[概念] community.general.lvg/lvol 管 VG/LV，community.general.filesystem 管签名，ansible.posix.mount 管 fstab 与当前挂载；实际 FQCN/字段以已安装 Collection 为准。

[概念] 容量可表达绝对值、增量或百分比。自动化应描述目标总量，避免每次运行都增加；扩容文件系统需要 resizefs 或专用步骤，XFS 不缩小。

[操作语义] 先 gather facts/lsblk 调查，但不从设备名模式推定空闲。模块 state present 只推进对应层；远端 pvs/vgs/lvs/blkid/findmnt/df 形成验收。

[知识专题] 设备安全不是模块自动判断

① [知识点] 设备安全不是模块自动判断

lvg 的 pvs 参数会初始化/加入设备；题目设备不确定时先 assert 事实和人工证据门。不要自动 wipefs/force。

② [知识点] 目标大小避免增量漂移

lvol size 写目标容量或百分比；使用扩容参数时确认模块幂等语义。每次 +1G 的命令式 shell 会持续增长。

③ [验证点] LVM、文件系统和挂载三层

lvs 变大不代表 df 变大；mount state mounted/ephemeral/present 的当前与 fstab 语义不同，按模块文档选择。

[Cheatsheet] 设备安全不是模块自动判断；目标大小避免增量漂移；LVM、文件系统和挂载三层

[操作专题] 用变量描述每主机布局

① [操作点] 用变量描述每主机布局

host_vars 定义 devices、vg、lv、size、fstype、mount；assert 设备键存在和尺寸条件。

② [操作点] 按依赖创建

lvg → lvol → filesystem → file mountpoint → mount。已有 filesystem 不重建；扩容使用 resizefs 并保留数据。

③ [验证点] 执行前后与幂等

limit 单主机，远端查对象/UUID/fstab/findmnt/df/数据；第二次无 changed，再扩全组。

[Cheatsheet] 用变量描述每主机布局；按依赖创建；执行前后与幂等

[诊断专题] VG 不存在或设备不存在

① [诊断点] VG 不存在或设备不存在

用 hostvars/Facts 与远端 lsblk 证明条件，fail/assert 给明确消息，不用 ignore_errors。

② [诊断点] LV 变大 df 不变

文件系统 resize 未完成；按类型执行模块 resize 或 xfs_growfs，不继续增加 LV。

③ [诊断点] mount 每次 changed

检查 src 是否稳定（UUID/设备路径）、opts 顺序、state 与 fstab 现状；不要用 shell mount。

[Cheatsheet] VG 不存在或设备不存在；LV 变大 df 不变；mount 每次 changed

[知识专题] 设备事实、容量单位与失败保护

① [知识点] ansible_devices 不是安全空闲证明

Facts 可给设备大小、分区和型号，但未必包含所有签名/LVM/挂载关系。真正初始化前仍需受控 lsblk/blkid/pvs 证据或题目明确保证。

② [参数点] 容量字符串保持题意单位

size: 2g、百分比和 resizefs 行为由当前 collection 模块定义。extent 题若模块不能精确表达，应计算并 assert 结果，不用近似容量悄悄改变目标。

③ [失败边界] 先 fail 再做破坏性任务

用 assert 检查设备变量存在、VG 条件和目标大小；block/rescue 可输出明确错误，但不能在 rescue 中格式化另一块猜测设备。

[Cheatsheet] Facts 只做初筛；容量保持题意单位并 assert；破坏性 task 前显式 fail，rescue 不猜替代设备。

[经典任务] 按主机变量创建并挂载逻辑卷

在指定主机使用给定空闲设备创建 VG/LV/ext4 或 XFS 并持久挂载；空间不足时输出明确失败，不得格式化未知设备。随后扩容并保留标记数据。验收模块结果、三层容量、fstab、挂载和幂等。

请先独立完成。参考解答从下一页开始。

[参考解答] 把设备差异放入变量并设置证据门

- ① [操作点] `host_vars` 描述布局, `assert` 设备/容量前提。
- ② [操作点] `lvg/lvol/filesystem/mount` 按依赖; 扩容 `resizefs`。
- ③ [验证点] `pvs/vgs/lvs`、`blkid`、`findmnt/df`、标记校验, 第二次执行。

[Cheatsheet] 变量/设备证据 → LVM → filesystem → mount → 数据 → 扩容 → 幂等。

[本章收束] 幂等模块不能替代设备风险判断

模块擅长比较已知对象状态, 但初始化哪个设备仍由数据和证据决定。按依赖创建并分层验收, 才能避免自动化放大破坏性错误。

第十一章 Ansible 故障排除、验证与幂等性

Ansible 故障首先归类为控制端解析、Inventory 匹配、SSH/提权、模块执行或远端终态。verbosity 和 register 提供证据；ignore_errors、无条件 changed_when 或 recap 绿色不能替代根因修复。

[概念] FAILED 表示主机可达但任务失败，UNREACHABLE 表示连接层未建立，SKIPPED 是条件未满足，CHANGED 是模块认为状态改变。它们是执行分类，不是业务结论。

[概念] 幂等性要求相同目标状态重复执行不再改变；正确性要求目标本身符合题意。错误配置也可能稳定幂等，因此两者必须组合。

[操作语义] 从 `--syntax-check/--list-hosts` 开始，使用 `-v/-vvv` 增加证据，debug 注册对象，`--step/--start-at-task` 谨慎缩小；远端协议与系统查询最终验收。

[知识专题] 按失败层分类

① [知识点] 按失败层分类

YAML/模块字段在控制端；0 hosts 在 Inventory；unreachable 在 SSH；become 在 sudo；failed msg 在模块/远端；绿色 recap 后仍查终态。

② [知识点] changed 语义可被模块与规则改写

changed_when 应基于可靠输出，不为追求全绿无条件 false；failed_when 不能吞掉真实失败。

③ [验证点] 第二次执行与远端状态并行

比较两次 recap/任务 diff，同时从受管节点查文件、服务、端口、挂载和数据。

[Cheatsheet] 按失败层分类；changed 语义可被模块与规则改写；第二次执行与远端状态并行

[操作专题] 最小化复现

① [操作点] 最小化复现

syntax/list/limit 单主机，使用 tags/start-at-task 前确认依赖；verbosity 只提升到能回答问题的层。

② [操作点] 读取 register 结构

debug var 完整结果，区分 rc/stdout/msg/results；用 assert 明确前置和验收条件。

③ [验证点] 修复后做全链回归

从失败层向下到远端功能，再全组执行和第二次执行；检查 Handler 是否按变化运行。

[Cheatsheet] 最小化复现；读取 register 结构；修复后做全链回归

[诊断专题] 变量未定义/模板错误

① [诊断点] 变量未定义/模板错误

查最终 hostvars、数据类型、模板路径和 scope；不要用 default 空值掩盖。

② [诊断点] unreachable 被当作 task 失败

业务模块尚未执行，先修 SSH/host key/remote_user/Python。

③ [诊断点] 每次 changed 但终态相同

查 command/shell、mtime、随机值、模板动态内容与 module 参数；不要全局 changed_when false。

[Cheatsheet] 变量未定义/模板错误；unreachable 被当作 task 失败；每次 changed 但终态相同

[知识专题] check mode、diff 与可观测性边界

① [知识点] check mode 支持度取决于模块

支持 check_mode 的模块预测 changed，不支持的任务可能跳过或仍需特殊处理。不能把一次 `--check` 全绿当作真实执行成功。

② [验证点] diff 只显示可公开差异

`--diff` 有助于模板/文件审查，但可能暴露敏感内容；秘密任务应 `no_log`，并用非敏感结构证据验收。二进制或某些模块没有可读 diff。

③ [诊断点] start-at-task 会跳过前置状态

从中间开始可能缺少 Facts、变量设置、文件下载或 Handler 通知，只适合前置状态已明确成立的复现。最终仍从完整 Playbook 回归。

[Cheatsheet] check 支持度要核对；diff 注意秘密；start-at-task 不替代完整回归。

[经典任务] 修复一个多层故障项目

项目同时存在错误 Inventory 组、变量类型、模板字段、Handler 名称和无条件 changed。要求逐层修复，不能 ignore_errors；最终全组终态正确且第二次无无法解释 changed。

请先独立完成。参考解答从下一页开始。

[参考解答] 按层收缩并回归

① [诊断点] syntax/list-hosts/inventory 先修控制与范围；ping/become 修连接。

② [诊断点] limit 运行，debug register/变量，修模板与 Handler。

③ [验证点] 远端文件/服务/端口，全组与第二次执行。

[Cheatsheet] 控制端 → Inventory → 连接/become → task/Handler → 远端终态 → 全量/幂等。

[本章收束] 绿色执行不是最终判据

可靠排错让每个错误回到所属层，可靠验收同时看执行语义和远端事实。幂等性是正确终态的附加条件，不是替代条件。

第十二章 RHCE 综合任务

RHCE 综合任务把项目配置、变量、Vault、Role 和系统状态组合为一套多节点声明。完成标准不是 Playbook 数量，而是所有目标主机被正确覆盖、秘密安全、远端终态正确且第二次执行稳定。

[概念] 控制面包含 `ansible.cfg`、Inventory、requirements、group_vars/host_vars、Vault、templates 和 roles；数据面是受管节点上的用户、包、服务、网络、安全和存储。

[概念] 执行应从静态边界到单主机，再到主机组和全量；每个阶段保留 recap 与远端证据。

[操作语义] 发布入口是可重复的构建/执行命令；`--syntax-check`、inventory graph、requirements install、vault access、limit/full/idempotence 和远端验收构成主链。

[知识专题] 先审计项目完整性

① [知识点] 先审计项目完整性

确认题目要求的文件路径、名称、权限、Vault 密文、Role/Collection 依赖和 Inventory 组均存在；不从任意工作目录执行。

② [知识点] 桥接 RHCSA 目标为模块 state

用户/包/服务/防火墙/SELinux/存储使用专用模块或 System Roles；shell 仅补明确缺口并定义 changed/failed。

③ [验证点] 每组主机有独立终态矩阵

web、db、balancer 等分别核对变量差异、文件、服务、端口和安全；inventory 漏主机不会在 recap 中自动报错。

[Cheatsheet] 先审计项目完整性；桥接 RHCSA 目标为模块 state；每组主机有独立终态矩阵

[操作专题] 执行前门

① [操作点] 执行前门

安装 requirements, vault view, syntax/list-hosts/list-tasks; ping/become; check/diff（支持范围）。

② [操作点] 从一台到全量

limit 代表主机，修复控制/连接/task 问题并验远端；扩组、全量，观察 Handler 与失败分类。

③ [验证点] 第二次执行和外部功能

再次全量，解释 changed；从客户端验证 HTTP/网络，从节点验证身份、包、服务、SELinux、mount 和数据。

[Cheatsheet] 执行前门；从一台到全量；第二次执行和外部功能

[诊断专题] 全绿但漏主机

① [诊断点] 全绿但漏主机

对照题目与 `--list-hosts` /Inventory graph，不能只看 recap 中出现的主机。

② [诊断点] 秘密或生成物泄露

检查 debug/no_log、明文密码文件权限、构建输出和版本控制；保持 Vault 加密。

③ [诊断点] 修一组破坏另一组

把差异移到变量/模板条件，回归所有共享 Role 消费者；不要复制分叉 Role。

[Cheatsheet] 全绿但漏主机；秘密或生成物泄露；修一组破坏另一组

[知识专题] 交付目录、命名与可重复执行入口

① [验证点] 题目要求的路径本身可能评分

Inventory、ansible.cfg、playbook、Vault、templates 和 roles 必须位于指定绝对路径并具备正确权限。内容等价但文件名或入口错误仍可能无法被评分命令发现。

② [知识点] 控制节点产物与受管节点产物分开

template 源、requirements 和 Vault 留在控制端；目标配置、用户和挂载在远端。验证脚本要明确在哪一端执行，不能看到控制端文件就判定远端完成。

③ [验证点] 最终执行记录可重现

从项目目录运行固定命令，记录 inventory graph、syntax、首次/第二次 recap 和远端矩阵。清理临时明文与 debug 任务，但不删除题目要求的内容源。

[Cheatsheet] 路径/命名是接口；控制端与远端对象分开；固定入口重现首次、二次与远端矩阵。

[经典任务] 完成一个多组主机自动化项目

建立指定 Inventory/配置、Vault 与 Role，配置用户、仓库、包、模板/Handler、网络、防火墙/SELinux、LVM/mount，并使用 requirements。不得泄露秘密或掩盖错误。验收文件路径、主机覆盖、各组终态、外部功能与第二次执行。

请先独立完成。参考解答从下一页开始。

[参考解答] 按发布门连续推进

- ① [操作点] 审计项目/依赖/Vault/Inventory, syntax/list/ping/become。
- ② [操作点] limit 代表主机，远端验收；扩组与全量。
- ③ [验证点] 全量第二次执行；按主机组矩阵查身份/服务/网络/安全/存储与外部功能。

[Cheatsheet] 项目完整性 → 静态/连接 → 单机 → 分组/全量 → 远端矩阵 → 幂等/安全审计。

[本章收束] 自动化交付是可复现的正确终态

项目源、依赖、秘密和执行边界必须可重建；受管节点的目标状态必须由专用模块表达并由外部证据验证；第二次执行则证明这种正确状态可以稳定维持。